

Flutter로 비전 앱 만들기 — 방법론

웹 우선 개발에서 네이티브 확장까지, 이 워크샵 앱이 실제로 거친 4단계

"Flutter 문법"이 아니라 "비전 앱을 어떤 순서로, 어떻게 검증하며 키웠는가"를 다룹니다.

강사: 김생근

2026년 7월 · 부록 — Flutter 방법론

왜 "Flutter 문법"이 아니라 "방법론"인가요?

Flutter 공식 튜토리얼은 위젯 문법·상태 관리를 가르칩니다. 이 문서는 그걸 대체하지 않습니다 — 대신 이 워크샵의 통합 비전 앱이 실제로 손 감지 데모 1개(a버전)에서 홈+6탭 통합 서비스 (d버전+통합)까지 자라는 동안, "무엇을 먼저 결정했고, 왜 그 순서였고, 어떻게 검증했는가"를 정리합니다.

이 순서를 모르면 흔히 벌어지는 일: 네이티브 앱부터 만들려다 Android Studio·Xcode 설치에 며칠을 쓰고, 아이디어 검증은 시작도 못 합니다. 또는 신규 기능을 추가할 때마다 기존 코드를 고쳐 회귀를 만듭니다. 또는 "빌드 성공"을 "작동 확인"으로 착각하고 배포합니다. 이 문서는 이 세 가지 함정을 실제로 겪고 피한 방법을 다룹니다.

학습 목표

#	이 문서를 마치면
1	비전 앱을 웹 우선(Web-first)으로 시작해야 하는 이유를 설명할 수 있다
2	계약 기반(Contract-first) 패턴으로 신규 기능을 기존 코드 훼손 없이 추가할 수 있다
3	"빌드 성공"과 "실제 작동 확인"을 구분하는 3단계 검증을 안다
4	네이티브(Android/iOS) 확장이 필요한 시점과 그때 필요한 도구를 안다

개념 1. 웹 우선(Web-first) — 네이티브 설치 없이 먼저 반응을 본다

🤔 왜 필요한가?

아이디어가 맞는지도 모르는 단계에서 Android Studio(수 GB) · Xcode(수십 GB) 설치와 스토어 심사 절차부터 밟으면, 정작 "이 손 감지 기능이 쓸모 있는가"라는 진짜 질문에 도달하기 전에 시간이 소진됩니다.

💡 비유로 이해하기

매장을 정식으로 열기 전에 팝업스토어로 먼저 반응을 보는 것과 같습니다. 팝업스토어(웹)는 URL 하나로 오늘 열고 오늘 피드백을 받습니다. 반응이 좋으면 그때 정식 매장(네이티브 앱)을 준비해도 늦지 않습니다.

🔍 원리 설명

이 워크샵 앱은 MediaPipe Tasks Vision JS(WASM 빌드)를 브라우저에 직접 로드해, 서버 없이 클라이언트 사이드에서 전부 처리합니다. Flutter 쪽은 "화면과 상태"만 맡고, 실제 카메라·추론은 JS가 처리한 결과를 dart:js_interop으로 받아옵니다.

공식 Flutter 카메라/MediaPipe 플러그인이 없거나(Android 전용이거나 버전이 안 맞는 경우가 흔함) 있어도 불안정할 때, "플러그인이 지원할 때까지 기다린다" 대신 "이미 검증된 JS 라이브러리를 웹 레이어에서 직접 부른다"는 선택입니다.

🤖 실무 관점

이 앱은 a(손 감지)→b(+얼굴)→c(+사물)→d(4탭+리디자인)까지 네 번 확장되는 동안 전부 웹 배포만으로 진행됐습니다. 매번 GitHub Pages URL 하나로 실제 브라우저에서 확인했고, 네이티브 빌드 절차는 단 한 번도 필요하지 않았습니다.

? 그럼 네이티브 앱은 아예 필요 없나요?

아닙니다 — "먼저 확인하고 나중에 정한다"는 순서의 문제입니다. 카메라 성능이 웹보다 훨씬 좋아야 하거나, 오프라인 동작·백그라운드 처리·스토어 배포가 실제로 필요해지는 시점에 네이티브

브로 넘어갑니다. 그 구체적 절차는 6절에서 다룹니다.

개념 2. 계약 기반(Contract-first) 확장 — 새 기능이 기존 코드를 안 건드리게

🤔 왜 필요한가?

탭이 1개에서 7개로 늘어나는 동안 매번 main.dart의 기존 로직을 고치면, 신규 기능 하나가 기존 3개 탭을 망가뜨릴 위험이 계속 쌓입니다.

💡 비유로 이해하기

전기 콘센트 규격과 같습니다. 새 가전을 살 때마다 벽 공사를 다시 하지 않습니다 — 규격(전압·플러그 모양)에 맞추기만 하면 꽂힙니다. 이 앱의 각 탭도 "이 모양(계약)만 지키면 꽂힌다"는 규칙으로 설계했습니다.

🔍 원리 설명

공용 타입 TabConfig{ label, icon, viewType, statusEvent, start, stop, actions }와 AppTab 리스트를 먼저 뽑아 두고(공용 계약 추출), 이후 신규 탭(마법지팡이·문서스캔 등)은 이 계약에 맞는 새 파일만 추가합니다. 기존 파일은 딱 한 곳 — 새 탭을 AppTab 리스트에 등록하는 부분만 수정합니다.

🤖 실무 관점

문서 스캔(OpenCV 기반)·마법 지팡이(색상 추적) 같은 완전히 다른 알고리즘의 탭도 이 계약만 지키면 되므로, 4개 신규 탭 작업을 서로 겹치지 않는 새 파일로 병렬 진행할 수 있었습니다 — 계약이 먼저 있어야 병렬 작업도 가능해집니다.

? 계약을 먼저 안 뽑고 그때그때 짜면 안 되나요?

되긴 합니다 — 다만 나중에 탭이 늘어날수록 "이 탭은 왜 저 탭이랑 구조가 다르지"라는 정리 비용이 커집니다. 이 앱은 실제로 4개 탭이 될 시점에 계약을 먼저 추출하고, 그 위에 새 탭을 얹는 순서로 리팩터링 비용을 줄였습니다.

계약이 먼저, 기능은 그 다음 — 순서를 바꾸면 매번 새로 설계하게 됩니다.

개념 3. 3단계 검증 — "빌드 성공"은 "작동 확인"이 아니다

🤔 왜 필요한가?

`flutter build web`이 에러 없이 끝났다는 것은 문법이 맞다는 뜻이지, 카메라가 실제로 켜지고 화면에 손이 제대로 그려진다는 뜻이 아닙니다. 이 둘을 같다고 착각하는 게 가장 흔한 실패 지점입니다.

💡 비유로 이해하기

자동차 출고 전 검사와 같습니다. 부품이 도면대로 조립됐는지(정적 분석), 엔진이 시동은 걸리는지(단위 테스트), 실제로 도로에서 몰아봤을 때 브레이크가 듣는지(실제 환경 검증) — 세 단계는 서로 다른 것을 확인하고, 앞 단계 통과가 뒷 단계를 보장하지 않습니다.

🔍 원리 설명

단계	확인하는 것	명령
① 정적 분석	문법·타입 오류	flutter analyze
② 단위/위젯 테스트	로직이 의도대로 동작하는가(카메라 없이)	flutter test
③ 실제 환경 검증	진짜 브라우저·웹캠에서 실제로 되는가	flutter test --platform chrome, Playwright

🤖 실무 관점

①②만 통과하고 "완료"라 부르면 안 됩니다. 이 프로젝트에서 실제로 겪은 사례: `prefers-reduced-motion` (모션 최소화 접근성 설정) 처리를 Flutter의 `MediaQuery.disableAnimations`로 구현했더니 코드는 정상 컴파일되고 테스트도 통과했지만, 실제 브라우저에서는 반영되지 않았습니다 — ③단계(Playwright로 실제 브라우저 확인)에서만 드러난 문제였고, JS `matchMedia` 브릿지로 바꿔서야 해결됐습니다.

? 정적 분석·단위 테스트가 다 초록불이면 충분한 거 아닌가요?

아니요. 이 프로젝트의 Flutter 스캐폴딩 파일(pubspec.yaml 등)이 한 번도 커밋되지 않은 채 방치된 적도 있었습니다 — 로컬에서는 전부 잘 도는 것처럼 보였지만, 다른 환경에서 그대로 clone하면 빌드조차 안 되는 상태였습니다.

내 화면에서 되는 것과, 다른 환경·실제 사용자 앞에서 되는 것은 다른 질문입니다.

이 앱을 네이티브(Android/iOS)로 확장한다면

지금 이 앱은 웹으로만 배포되어 있습니다(android/ios 폴더 자체가 없습니다). 네이티브 확장이 실제로 필요해지는 시점 — 오프라인 동작, 카메라 성능, 스토어 배포가 요구될 때 — 아래가 필요합니다.

플랫폼	필요한 도구	비고
Android	Android Studio (Android SDK 포함)	Windows·macOS·Linux 어디서나 설치 가능
iOS	Xcode	macOS에서만 설치·빌드 가능 (애플 정책) + 실기기 배포·스토어 등록엔 Apple Developer Program(연 \$99) 별도 필요

설치 후에는 기존 코드를 그대로 두고 플랫폼 폴더만 추가합니다 — 새로 만드는 게 아니라 "덧붙이는" 명령입니다.

🚧 플랫폼 추가 명령

```
cd src/part3_flutter_web_d

flutter create --platforms=android . # android/ 폴더 추가

flutter create --platforms=ios .     # ios/ 폴더 추가 (Mac 전용)

flutter build apk                    # 안드로이드 설치 파일 생성

flutter build ios                    # iOS 빌드(시뮬레이터/서명 없는 수준까지)
```

자세한 단계별 안내와 이 컴퓨터의 실제 환경 진단 결과는 [doc/tutorials/flutter.md](#)에 정리되어 있습니다.

1~2분 요약

1~2분 요약 — 이것만 기억하세요

1. 웹 우선: 아이디어를 검증할 땐 네이티브 설치·심사 전에 웹으로 먼저 반응을 본다.
2. 계약 기반: 신규 기능을 추가하기 전에 공용 타입(계약)을 먼저 뽑아 두면, 이후 기능은 기존 코드를 안 건드리고 새 파일만으로 붙는다.
3. 3단계 검증: 정적 분석 → 단위 테스트 → 실제 브라우저/웹캠 확인. 앞 두 단계 통과가 세 번째 단계를 보장하지 않는다.

**"빌드가 성공했다"와 "실제로 작동한다"는 다른 문장입니다 — 이 문서 전체가 그 차이를 매우
는 방법입니다.**

빠른 참조 카드

단계별 체크리스트

단계	확인할 것
시작 전	네이티브 도구 설치보다 flutter run -d chrome으로 먼저 되는지 확인했는가
기능 추가 전	기존 코드를 고치지 않고 새 파일만으로 계약에 맞출 수 있는가
"완료" 선언 전	정적 분석·단위 테스트뿐 아니라 실제 브라우저에서 확인했는가
네이티브 전환 시	Android는 Android Studio, iOS는 Mac+Xcode(+유료 개발자 계정)가 준비됐는가

이 앱의 웹-네이티브 확장 상태

플랫폼	상태
Web	✅ 배포됨 — https://kimsanguine.github.io/vision-vibe/
Android	❌ 미착수 — android/ 폴더 없음, Android Studio 필요
iOS	❌ 미착수 — ios/ 폴더 없음, Mac+Xcode 필요

출처: 이 워크샵 *src/part3_flutter_web_a~d/TUTORIAL.md, doc/tutorials/flutter.md (2026-07-31 기준)*